

AI workflow guide

How to use the agents, slash commands, and skills in this project, and how to prompt them well.

Copy the `workspace/` bundle into the project root; it adds `.github/` (agents, prompts, skills, instructions), `.vscode/extensions.json`, and this guide.

Set up

1. Copy the bundle contents into the project root (merge `.vscode/extensions.json` if one exists).
2. Install the recommended extensions: draw.io (`hediet.vscode-drawio`) and Markdown Mermaid.
3. Open Chat. Check the Agent dropdown lists Quick Helper, Planner, Implementer, Deep Researcher, and Orchestrator. If one is missing, right-click the Chat view, choose Diagnostics, and read the errors.
4. Model names are pinned in each `.agent.md` (`model:`). If a name does not match your plan, the next one in the list is used. See the `model-routing` skill to change them.

Pick the agent

You want to	Use	Model tier
Fix one thing in one file, ask a quick question	<code>/quick</code>	cheap and fast
Design a feature, plan a project, review a layout	<code>/plan</code>	balanced, read-only
Build an approved plan	<code>/act</code>	strong coder, edits and runs tests
Find a root cause, evaluate a hard trade-off	<code>/deep-research</code>	deep reasoning, read-only
Not sure which, or want plan-then-build handled for you	Orchestrator	routes automatically

Default flow for anything bigger than a small edit: `/plan`, `approve`, `/act`.

Skills

Skills load on their own when your request matches their description. You can also name one in the prompt (use the `writing-mermaid` skill) or run it as a slash command (`/planning-projects`).

Skill	Ask for it like this
<code>researching-ideas</code>	"Compare Redis, Postgres LISTEN/NOTIFY, and RabbitMQ for our job queue. Constraints: one VPS, small team. Give a recommendation and what would change it."
<code>planning-projects</code>	"Plan an MVP for a club-membership app: .NET API, Angular web, Postgres. Must-haves: sign-up, payments, member list. Hosting: Hetzner VPS."
<code>modeling-uml</code>	"Reverse-engineer a class diagram of the Billing domain from <code>#file:src/Billing.Domain/Invoice.cs</code> and its neighbors. Public members only."

Skill	Ask for it like this
writing-mermaid	"Mermaid sequence diagram for login with refresh token: browser, API, auth service, database. Put it in <code>docs/auth.md</code> ."
designing-diagrams	"Make a draw.io container diagram of the whole system for new developers: web, mobile, API, worker, Postgres, S3. Save to <code>docs/diagrams/system.drawio</code> ."
model-routing	"Which model should I use to migrate 40 files from Moment to date-fns?"

Choosing between the diagram skills: Mermaid for quick, versioned diagrams inside Markdown (sequence, state, ER, small flows); draw.io for polished, editable architecture and class diagrams; `modeling-uml` when you need the right UML type and notation, drawn in either.

How to prompt well

Give five things; skip any that are obvious from the code.

1. **Goal:** the outcome, in one sentence.
2. **Context:** where to look (`#file:path`, folder, ticket text) and what already exists.
3. **Constraints:** stack, patterns to follow, things not to touch, size limits.
4. **Done when:** the check that proves it works (`dotnet test`, a URL, a screenshot).
5. **Output:** the shape you want back (checklist, table, diff, 10-line summary).

Weak: `fix the login bug`

Strong: `Login returns 500 when the email has uppercase letters. Look in #file:src/Api/Auth/LoginHandler.cs and its tests. Don't change the public API. Done when a new test for mixed-case emails passes with dotnet test. Reply with the cause in two lines and the diff.`

Habits that save time and tokens

- **One task per chat.** Start a new chat when the topic changes. For `/act` after a long `/plan` exploration, open a new chat and paste only the approved checklist.
- **Point at files** instead of describing them. Attach with `#file:path` or drag them into Chat.
- **Ask for options with a recommendation** when you are deciding: "Give 3 options, recommend one."
- **Say what not to do:** "Don't refactor", "Don't add dependencies", "Keep the existing naming."
- **Trim pasted output.** Paste the failing lines and the command, not a 500-line log.
- **Approve plans explicitly.** Reply "approved" or edit the checklist; Implementer will not start otherwise.
- **After two failed fixes, switch to `/deep-research`** with the symptoms and what you tried.
- **Ask for sources and confidence** on research: "Cite primary sources and mark claims verified or unverified."
- **Review small slices.** Ask Implementer to stop after each checklist item when the change is risky.

Prompt templates

Feature:

```
/plan Add CSV export to the Orders page.  
Context: #file:src/Orders/OrdersController.cs, Angular page in src/app/orders.  
Constraints: stream large exports, no new packages.  
Done when: a 100k-row export finishes without memory growth.  
Output: checklist with file paths and test commands.
```

Bug:

```
/deep-research Nightly sync intermittently duplicates rows.  
Symptoms: duplicates only after a timeout retry. Tried: added a unique index  
(migration failed on prod data).  
Logs: <last 20 relevant lines>.  
Output: root cause with confidence, ruled-out causes, recommended fix.
```

Idea:

```
Use researching-ideas: should we move from Docker Compose on one VPS to  
Kubernetes?  
Team of 2, 3 services, ~200 req/s peak. Output: recommendation, risks, what would  
change it.
```

Docs:

```
Use modeling-uml and writing-mermaid: class diagram of the Domain project, public  
members only,  
saved in docs/domain.md.
```

Recipes

- **New project:** `/plan` with `planning-projects`, approve, `/act` phase 1, then ask for an architecture diagram with `designing-diagrams`.
- **Evaluate an idea:** `researching-ideas`; if the answer is close or high-stakes, rerun with `/deep-research`.
- **Hard bug:** reproduce, `/deep-research`, `/plan` the fix, `/act`.
- **Document a system:** `modeling-uml` for structure, `writing-mermaid` for flows, `designing-diagrams` for the big picture.

Troubleshooting

- **A slash command is missing or conflicts with a built-in one:** change `name:` in the file under `.github/prompts/`, or pick the agent from the dropdown instead.
- **A skill did not trigger:** name it in the prompt, or run it as `/skill-name`.

- **Wrong model used:** check the agent's `model:` list and your plan's available models, then see Diagnostics in the Chat view's context menu.
- **A diagram will not open or render:** run `.github/skills/designing-diagrams/scripts/validate-drawio.ps1 <file>` for draw.io files; for Mermaid, fix the first error shown in the Markdown preview.
- **Responses ignore your rules:** keep `AGENTS.md` and `.github/copilot-instructions.md` short and stable; put task-specific detail in the prompt.